

Cursor AI Training

AI-Assisted Software Engineering with Cursor AI

Advanced Agentic Engineering Track

Total Duration: 32 Hours — 8 Half-Days (4 hours / half-day)

1. Agenda / Goal

- Provide developers with a comprehensive, engineering-lifecycle view of AI-assisted software engineering, using Cursor AI as the primary IDE and agentic orchestration platform
- Ground the course in GenAI/LLM fundamentals before introducing any tool-specific mechanics, so participants understand why the tooling works the way it does
- Enable teams to adopt AI-assisted and multi-agent coding practices across the full SDLC — design, development, testing, review, deployment, and maintenance — not just code generation
- Cover core Cursor AI features: AI chat, inline code generation, codebase-aware editing, refactoring, debugging, and custom rules
- Introduce Spec-Driven Development (SDD): treating a structured, versioned spec as the durable source of truth an agent implements against, with full spec-to-code traceability
- Build reusable agent, prompt-template, and skills frameworks grounded in repository code, requirement documents, and standards
- Establish multi-agent orchestration patterns with automated quality gates, correction loops, and human approval checkpoints
- Instrument agent pipelines for observability, and build fluency in token economics and ROI modeling to support enterprise rollout decisions
- Apply the full stack to transform a ticket into a validated, independently reviewed, deployment-ready, traceable engineering report
- Build hands-on proficiency through five progressively advanced use-case labs, culminating in an end-to-end capstone

2. Prerequisites

- Working knowledge of at least one programming language (JavaScript/Python/Java/.Net preferred)
- Familiarity with Git and version control workflows
- Basic understanding of IDEs (VS Code experience helpful, not mandatory)
- Understanding of the team's existing codebase/repo structure and SRS/SDS documentation practices
- Basic familiarity with a ticketing tool (Jira or Azure DevOps) — used in capstone
- Basic familiarity with unit/API testing concepts (PyTest or equivalent helpful, not mandatory)
- Basic familiarity with software design artifacts (architecture diagrams, API contracts) — used in the Spec-Driven Development module
- Basic understanding on AI/LLM or multi-agent orchestration concepts

3. Software / System / License Requirements

- Cursor AI – latest stable version, Enterprise/Team seat with Agent Mode enabled (Windows/Mac/Linux)
- System requirements: Minimum 8GB RAM (16GB recommended), 5GB free disk space
- Git installed and configured; access to a sandbox/training repository (provided by facilitator)
- Node.js / Python (or relevant language runtime) installed, based on project stack
- PyTest (or equivalent test framework) installed for the test-generation and validation labs
- Sample SRS/SDS documents, spec/SDD templates, coding standards, and a historical defect log (sandbox dataset provided)

- MCP server access configured for repository, documentation, and knowledge-graph/vector-store connections (sandbox instance provided — pre-stage before Half-Day 3)
- Read-access API token / sandbox project for Jira or Azure DevOps (provided by facilitator) for the capstone
- CI/CD sandbox (e.g., GitHub Actions) for the deployment-readiness portion of Module 19
- Stable internet connection (required for AI model and orchestration calls)
- Company SSO/IT approval for Cursor AI and connected MCP/agent tooling (if applicable)

4. Module Overview — 8 Half-Day Schedule

Day 1 — Foundations — GenAI/LLM Refresher & Cursor AI Introduction

#	Module / Segment	Duration
0	Introduction and Pre-Assessment	60 mins
1	GenAI/LLM Foundations Refresher	60 mins
—	Break	15 mins
2	Introduction to Cursor AI	30 mins
3	Cursor Interface & Setup	30 mins
4	AI Chat & Contextual Assistance	45 mins
	Day 1 total	240 mins

Day 2 — Core AI-Assisted Coding

#	Module / Segment	Duration
5	Inline Code Generation & Autocomplete	45 mins
6	Codebase-Aware Editing (Multi-file Context)	60 mins
—	Break	15 mins
7	AI-Assisted Refactoring & Debugging	60 mins
8	Custom Rules, Prompts & Team Standards	60 mins
	Day 2 total	240 mins

Day 3 — Design, SDD & Agent Architecture

#	Module / Segment	Duration
9	AI-Assisted Design & Spec-Driven Development (SDD)	90 mins
—	Break	15 mins
10	Agent & Skill Architecture Fundamentals	45 mins
11	Use Case Lab 1: Reusable Agents, Prompts & Skills Framework	90 mins
	Day 3 total	240 mins

Day 4 — Knowledge Grounding, Governance & Observability

#	Module / Segment	Duration
12	Knowledge Grounding Fundamentals - MCP Overview	90 mins
—	Break	15 mins
13	Use Case Lab 2: Knowledge-Grounded Engineering Agent	90 mins
14	Governance, Security & Observability	45 mins
	Day 4 total	240 mins

Day 5 — Multi-Agent Orchestration Build

#	Module / Segment	Duration
15	Orchestration Patterns Overview	45 mins
16	Use Case Lab 3: Multi-Agent Requirement-to-Test Automation	60 mins
—	Break	15 mins
16	Use Case Lab 3: Multi-Agent Requirement-to-Test Automation (cont..)	90 mins
17	Quality Gates & Self-Correction Fundamentals	30 mins
	Day 5 total	240 mins

Day 6 — Self-Correction & Deployment Readiness

#	Module / Segment	Duration
18	Use Case Lab 4: Self-Correcting Agent Orchestration with Quality Gates	120 mins
—	Break	15 mins
19	Git, CI/CD & Ticketing Integration for Agentic Workflows	90 mins
—	Capstone project intro	15 mins
	Day 6 total	240 mins

Day 7 — Capstone Build

#	Module / Segment	Duration
—	Capstone kickoff & scoping	15 mins
20	Use Case Lab 5 (Capstone): End-to-End Ticket-to-Report Engineering Copilot	210 mins
—	Break	15 mins
	Day 7 total	240 mins

Day 8 — Review, Demos & Program Wrap-Up

#	Module / Segment	Duration
—	Peer / AI-assisted review of capstone	45 mins
—	Demos & presentations	75 mins
21	Best Practices, Enterprise Rollout & ROI	60 mins
—	Wrap-up, feedback & Q&A	30 mins
—	Break / Buffer / final polish	30 mins
	Day 8 total	240 mins

Total Duration: 32 hours (1,920 minutes) across 8 half-days.

5. Modules — Detailed Topics & Subtopics

Day 1 — Foundations: GenAI/LLM Refresher & Cursor AI Introduction

Module 1 — GenAI/LLM Foundations Refresher

Topics:

- What LLMs are and how they generate text (high-level, non-mathematical)
- Tokens, context windows, and why they constrain agent and tool design
- Prompting fundamentals: instruction clarity, few-shot framing, system vs. user prompts
- Capabilities and known limitations of generative AI (hallucination, knowledge cutoffs, non-determinism)
- How generative AI differs from traditional deterministic software
- The AI-assisted SDLC landscape — from single-prompt coding to agents to multi-agent orchestration

Subtopics:

- Brief landscape overview: major model families, IDE-integrated vs. chat-based tools
- Core terminology (LLM, context window, agent, orchestration, MCP, spec-driven development)

Hands-on: guided comparison of a raw LLM chat response vs. a codebase-grounded agent response — establishes why the rest of the programme is structured around grounding and agents.

Module 2 — Introduction to Cursor AI

Topics:

- What is Cursor AI and how it differs from traditional IDEs
- Cursor vs. GitHub Copilot vs. other AI coding tools
- Overview of AI-assisted SDLC — from single-prompt coding to multi-agent orchestration

Subtopics:

- Positioning Cursor within the AI-Assisted Software Engineering theme established in Module 1

Hands-on: quick tour and account/license verification.

Module 3 — Cursor Interface & Setup

Topics:

- Installing/configuring Cursor AI; connecting the enterprise license/account
- Interface walkthrough (Editor, Sidebar, Command Palette, Chat/Agent Panel)
- Importing existing VS Code settings/extensions; workspace and project setup

Subtopics:

- Connecting MCP servers (repository, docs, ticketing) at the workspace level

Hands-on exercise to follow this module.

Module 4 — AI Chat & Contextual Assistance

Topics:

- Using the AI Chat panel for code queries; context modes (file/folder/codebase-level)
- Using @-mentions to reference files, docs, or symbols

- Difference between Chat mode and Composer/Agent mode

Subtopics:

- Effective prompt writing; managing conversation history and context limits

Hands-on exercise and quiz to follow this module.

Day 2 — Core AI-Assisted Coding

Module 5 — Inline Code Generation & Autocomplete

Topics:

- AI-powered autocomplete (Tab-to-complete); inline generation via natural-language prompts
- Generating functions, classes, and boilerplate code; accepting, rejecting, and editing suggestions

Subtopics:

- Context-aware suggestions based on open files; customizing autocomplete settings

Hands-on exercise to follow this module.

Module 6 — Codebase-Aware Editing (Multi-file Context)

Topics:

- Using Composer/Agent mode for multi-file changes; understanding codebase indexing
- Making cross-file edits and feature implementations via AI; reviewing and approving diffs

Subtopics:

- Handling context window limitations in large projects; scoping AI edits safely

Hands-on exercise to follow this module.

Module 7 — AI-Assisted Refactoring & Debugging

Topics:

- Refactoring code using natural-language instructions; handling legacy code
- Using AI to identify bugs, explain stack traces, and suggest/apply fixes
- Writing and debugging unit tests with AI assistance

Subtopics:

- Validating AI refactors against existing tests; using terminal output as AI context

Hands-on exercise and quiz to follow this module.

Module 8 — Custom Rules, Prompts & Team Standards

Topics:

- Setting up .cursorrules / project-level AI instructions
- Enforcing coding standards via AI rules; creating reusable prompt templates
- Standardizing AI usage across the development team

Subtopics:

- Team-wide rule configuration and version control

Hands-on exercise to follow this module. Direct foundation for Module 9 (Spec-Driven Development) and Module 11 (reusable agent framework lab).

Day 3 — Design, Spec-Driven Development & Agent Architecture

Module 9 — AI-Assisted Design & Spec-Driven Development (SDD)

Topics:

- Writing a spec as the durable source of truth for agent-generated code
- Spec-to-code traceability; spec versioning and change management
- Contrasting Spec-Driven Development with prompt-driven and vibe-coding approaches
- Architecture diagram generation from requirements; API/interface contract drafting
- Architecture Decision Records (ADRs); translating SRS → SDS with AI assistance

Subtopics:

- What belongs in a spec vs. a prompt; acceptance criteria as testable spec clauses

Hands-on exercise: draft a spec for a sample feature and generate a first implementation pass against it.

Module 10 — Agent & Skill Architecture Fundamentals

Topics:

- Agent vs. rule vs. prompt vs. skill: definitions and when to use each
- Anatomy of a reusable agent (role, inputs, tools, guardrails, outputs)
- Parameterized prompt-template design
- Repository-level rules vs. team-level skills libraries

Subtopics:

- Versioning and reviewing agent/skill definitions like code

Hands-on walkthrough of a sample agent definition.

Module 11 — Use Case Lab 1: Reusable Agents, Prompts & Skills Framework

Lab objective: Create reusable engineering agents, parameterized prompt templates, repository rules, and skills for requirement analysis, test generation, validation, and documentation.

Activities:

- Define agent roles for requirement analysis, test generation, validation, and documentation
- Build parameterized prompt templates with explicit input/output contracts
- Package rules and skills into a shared, version-controlled library
- Test each agent independently against sample inputs

Deliverable: A reusable agent/skill library committed to the shared training repository.

Hands-on lab and peer review.

Day 4 — Knowledge Grounding, Governance & Observability

Module 12 — Knowledge Grounding Fundamentals

Topics:

- Grounding sources: repository code, SRS/SDS documents, standards, historical defects, knowledge-graph relationships
- Retrieval strategies: semantic search, structured lookups, graph traversal
- Hallucination risk in engineering contexts and why it matters

Subtopics:

- Designing "no unsupported facts" guardrails; citing sources in agent output

Hands-on walkthrough of a grounding configuration.

Module 13 — Use Case Lab 2: Knowledge-Grounded Engineering Agent

Lab objective: Build an agent that uses repository code, SRS/SDS documents, standards, historical defects, and knowledge-graph relationships without inventing unsupported engineering facts.

Activities:

- Connect the agent to repository, document, and knowledge-graph sources via MCP
- Configure retrieval scoping and source citation in agent responses
- Stress-test with out-of-scope questions to verify refusal/grounding behavior

Deliverable: A grounded agent that cites its sources and declines unsupported claims.

Hands-on lab and quiz.

Module 14 — Governance, Security & Observability

Topics:

- Data privacy and IP handling for connected documents and knowledge graphs
- Enterprise admin controls, audit logs, and access scoping for agent tool use
- Compliance considerations (SOC2, GDPR) as applied to grounded agents
- Tracing and logging across multi-agent pipelines
- What to log at each agent handoff for debuggability
- Cost visibility per agent call

Subtopics:

- Approved vs. restricted knowledge sources; restricted file/folder access policies
- Observability tooling options for agent pipelines

Hands-on exercise and quiz to follow this module.

Day 5 — Multi-Agent Orchestration Build

Module 15 — Orchestration Patterns Overview

Topics:

- Sequential vs. parallel multi-agent pipelines; state and handoff between agents
- Designing agent roles for a pipeline (Validator, Builder, Generator, Reviewer)
- Failure handling and retries in orchestration

Subtopics:

- Choosing orchestration granularity; observability/logging across agent handoffs (ties back to Module 14)

Hands-on walkthrough of a sample pipeline diagram.

Module 16 — Use Case Lab 3: Multi-Agent Requirement-to-Test Automation

Lab objective: Orchestrate Requirement Validator, Sequence Builder, Test Generator, API Validator, and Reviewer agents to convert an engineering requirement into validated executable tests.

Activities:

- Configure the Requirement Validator agent to check requirement completeness and testability
- Chain the Sequence Builder agent to derive a test sequence from the validated requirement
- Chain the Test Generator agent to produce executable tests (e.g., PyTest) from the sequence
- Chain the API Validator agent to check generated tests against the target API/interface
- Chain the Reviewer agent to sign off before tests are committed

Deliverable: An executable, validated test suite traced back to its source requirement.

Hands-on lab.

Module 17 — Quality Gates & Self-Correction Fundamentals

Topics:

- Designing automated PASS/FAIL criteria between agent stages
- Correction-loop patterns: who retries, what gets reused, what reruns downstream
- Human-in-the-loop approval checkpoints: where and why to place them

Subtopics:

- Avoiding infinite correction loops; logging gate decisions for traceability

Hands-on walkthrough of a gate configuration.

Day 6 — Self-Correction & Deployment Readiness

Module 18 — Use Case Lab 4: Self-Correcting Agent Orchestration with Quality Gates

Lab objective: Implement generator–validator–reviewer orchestration with automated PASS/FAIL gates, correction loops, downstream reruns, and human approval checkpoints.

Activities:

- Add automated gates between the generator, validator, and reviewer agents from Lab 3
- Implement a correction loop that routes FAIL results back to the generator with feedback
- Configure downstream reruns so only affected stages re-execute
- Add a human approval checkpoint before final sign-off

Deliverable: A self-correcting pipeline with gate logs and an approval trail.

Hands-on lab and quiz.

Module 19 — Git, CI/CD & Ticketing Integration for Agentic Workflows (EXPANDED)

Topics:

- Connecting Cursor AI/agents to Jira or Azure DevOps via MCP
- Parsing ticket fields into structured agent inputs
- Maintaining traceability from ticket to commit to test to report
- GitHub Actions pipeline basics; AI-assisted PR generation
- Automated deployment-readiness gates

Subtopics:

- Git workflow integration (branches, commits, PRs) for agent-generated artifacts

Hands-on exercise to follow this module.

Day 7 — Capstone Build

Module 20 — Use Case Lab 5 (Capstone): End-to-End Ticket-to-Report Engineering Copilot (EXPANDED)

Lab objective: Transform a Jira/Azure DevOps ticket into a requirement bundle, a spec note, an HIL test sequence, PyTest cases, a validation report, an independent review, a deployment-readiness check, and a traceable final engineering report with an observability/cost summary.

Activities:

- Pull a sandbox ticket and extract a structured requirement bundle
- Draft a short spec note establishing acceptance criteria (per Module 9's SDD approach)
- Generate the HIL (human/hardware-in-the-loop) test sequence from the requirement bundle
- Generate PyTest cases and execute them against the sandbox target
- Produce a validation report and route it to an independent reviewer agent
- Run the deployment-readiness gate from Module 19

- Compile a checklist and a traceable final engineering report linking ticket → spec → sequence → tests → results → review → deployment readiness, with a token-cost/observability summary

Deliverable: A complete, traceable engineering report package, ready for stakeholder sign-off.

Hands-on capstone build.

Day 8 — Review, Demos & Program Wrap-Up

Module 21 — Best Practices, Enterprise Rollout & ROI

Topics:

- Do's and don'ts of multi-agent AI-assisted development; avoiding over-reliance
- Verifying AI/agent output for correctness, security, and traceability
- Rolling out agent/skill libraries and quality gates across a team
- Token economics: cost-per-call and cost-per-task modeling; context-window cost implications of large codebase grounding
- Building an ROI case: time saved vs. token spend vs. defect reduction
- Using historical defect logs and production feedback to maintain and update agent grounding sources over time

Subtopics:

- Common pitfalls in orchestration and how to avoid them
- Maintenance and monitoring as an ongoing lifecycle phase, not a one-time handoff

Hands-on discussion.

Peer / AI-assisted review, demos & wrap-up: peer/AI-assisted review of each capstone deliverable against the rubric; demos and presentations of each team's end-to-end engineering report; wrap-up, feedback & Q&A.